POLITECNICO
MILANO 1863

AstraLog-HPC

ver.0.0(14.Apr.)

Daniela Hu, Junkai Ji, Emiliano Liao

March 2026

Contents

1 The project and project goals

AstroLog-HPC, the software that we meant to develop, has focus on anomalies detection. Due to myriad of telemetry data produced by the sensors installed on aircrafts, these last require a more efficient and suitable subsystem to avoid potential disasters.

As operative hypothesis, we assume the following characteristics:

- All data are directly streamd from sensors in the form of "**JSON packet**" according to the struture of the example below:

  {
  "timestamp": "2025-11-15T12:00:00Z",
  "sensor_id": "TEMP-01",
  "value": 25.5,
  "priority": "HIGH"
  }

- A unified synchronous system clock with measurements across all sensors equally spaced in time and perfectly aligned, e.g.: all sensors generate data exactly **one** measurement per second for the whole working time without any interruptions and, considering the matrix of measurements where the rows are the order of the measurement and the column the sensors, the datas of the same row have the exact same timestamp.

- The sensors are of several types, such as temperature and pressure, and their quantities are not necessary unitary. For further specification, the details are contained in dedicated file calles"**sensors.yaml**" and will not change during the spatial operation.

As stated above, the aim of these work is to recognize the anomalies through the regular examination of the obtained datas, therefore, we have also the type of errors to deal during the stellar project.

- Malformed JSON: truncated strings or invalid JSON syntax.

- Schema Errors: packets missing mandatory fields (e.g., missing sensor_id).

- Type Errors: invalid data types (e.g., a string instead of a numerical value).

The check of the datas are done using the rules, which are also given with a JSON file. There are several types of the rules and there is four fixed for them, here below are listed the commonly shared field:

- "rule_id": distinctive code of the rule. The form is "RX", where X is a number.

- "type": the "name" of the rule, it's also a brief description of the rule.

- "priority": a enum of "HIGH, MEDIUM, LOW". Such priority applies within a single rule type and means that rules with higher priority are evaluated before the others.

2 Requirement analysis

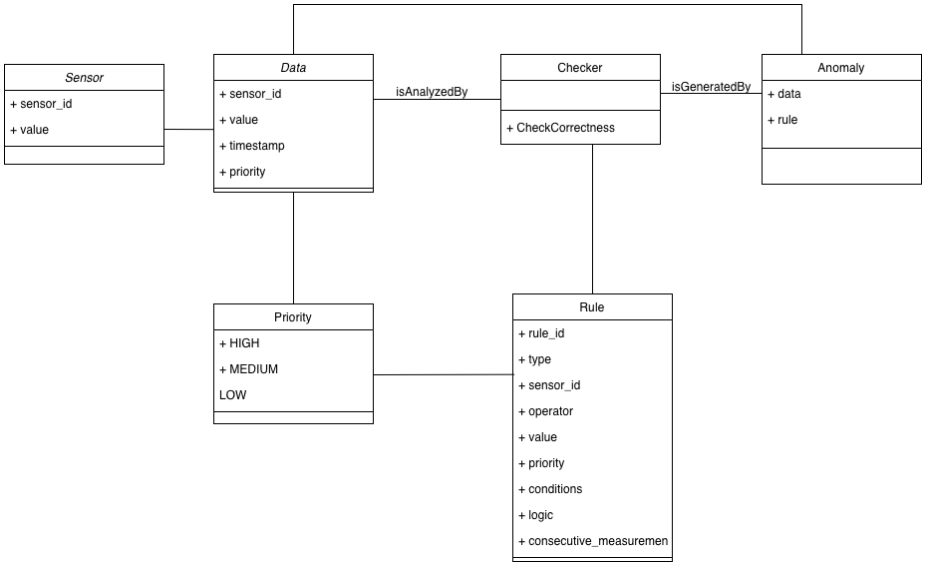

Figure 1: Class Diagram

2.1 Relevant human and non-human actors

2.1.1 Non-human actors

- **Sensors**: The primary data sources that generate JSON packets containing measurements such as temperature and pressure.
- **System Clock**: Provides a unified time reference for all sensors, ensuring that measurements are perfectly aligned in time.

2.1.2 Human actors

- **System Administrator**: Responsible for configuring the system, including setting up the sensors and defining the rules for anomaly detection.
- **Mission operator**: Monitors the system's output, reviews anomaly reports, and takes necessary actions based on the detected anomalies.

2.2 Use cases

- **Ingest telemetry stream**: The system receives JSON packets from the sensors in real-time.

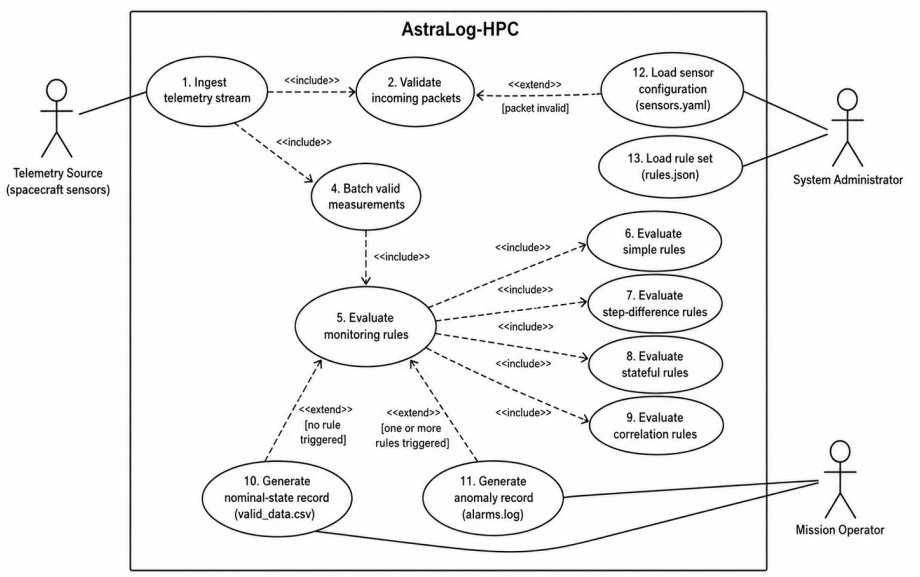

Figure 2: Use Cases

- **Validate incoming packets**: The system checks the structure and content of the incoming JSON packets against a predefined schema.
- **Batch valid measurements**: Validated packets are grouped into batches for efficient processing.
- **Evaluate monitoring rules**: The system applies predefined rules to the batches of data to identify any anomalies.
- **Evaluate simple rule**: The system evaluates simple rule that apply to individual measurements, such as checks if a value exceeds a fixed threshold at a given timestamp.
- **Evaluate step difference rule**: The system evaluates rules that compare current measurements with previous ones, such as checking if the difference between the current value and the previous value exceeds a certain threshold.
- **Evaluate stateful rule**: The system evaluates rules that consider the historical context of the data, such as checking if a value remains constant over a period of time.
- **Evaluate corelation rule**: The system evaluates rules that involve logical correlations between different measurements, such as checking if a high temperature coincides with a low pressure.

- **Generate nominal-state records**: If the data passes all the checks, the system generates records indicating that the measurements are within normal parameters.
- **Generate anomaly records**: If any of the rules are violated, the system generates detailed anomaly records for further analysis and reporting.
- **Load sensor configuration**: The system loads the sensor configuration from a YAML file, which includes details about the types and quantities of sensors.
- **Load rule set**: The system loads the set of monitoring rules from a JSON file, which includes the definitions and priorities of the rules to be applied during the evaluation process.

2.3 Domain assumptions

- **Unified Time Synchronization**: All sensors are synchronized to a unified system clock, ensuring that measurements across all sensors are equally spaced in time and perfectly aligned.
- **Consistent Data Format**: All data generated by the sensors adhere to a specific JSON structure, which includes mandatory fields such as timestamp, sensor_id, value, and priority.
- **Static Sensor Configuration**: The types and quantities of sensors are defined in a configuration file (sensors.yaml) and do not change during the operation of the system.
- **Rule-Based Anomaly Detection**: The system relies on a predefined set of rules, provided in a JSON file, to evaluate the data and identify anomalies. These rules are static and do not change during the operation of the system.

2.4 Requirements

2.4.1 Functional requirements

- **Data Ingestion**: The system must be able to receive and store JSON packets from the sensors in real-time.
- **Rule Evaluation**: The system must evaluate the ingested data against the defined rules and identify any anomalies based on the priority of the rules.
- **Anomaly Reporting**: The system must generate detailed reports for any detected anomalies, including relevant information such as sensor ID, timestamp, and violated rule.
- **Batch Processing**: The system must be able to process data in batches to optimize performance and efficiency.

2.4.2 Non functional requirements

- **Performance**: The system should be able to process incoming data and evaluate it against the rules in a timely manner, ensuring that anomalies are detected as soon as possible.
- **Reliability**: The system should be robust and able to handle potential errors in the data, such as malformed JSON or missing fields, without crashing or losing data.
- **Maintainability**: The system should be designed with modularity and clear documentation to facilitate future updates and maintenance.
- **Availability**: The system should be available and responsive under typical usage scenarios, minimizing downtime and ensuring continuous operation.

3 Design

3.1 Architectural Overview

AstraLog-HPC utilizes a **Pipe-and-Filter** architecture. This pattern treats data as a continuous flow passing through a series of discrete, independent processing "filters" connected by "pipes." Each filter performs a specific transformation or validation task, where the output of one filter serves as the direct input to the next.

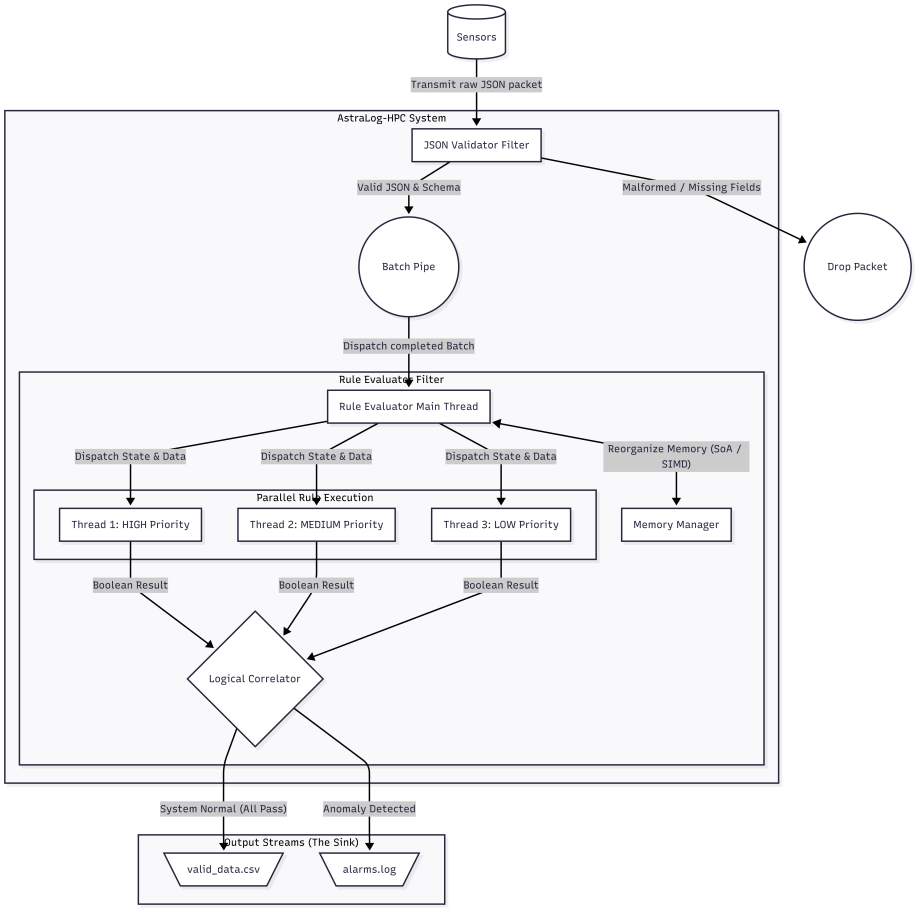

Figure 3: Component Diagram: Pipeline flow from Sensor ingestion to Rule Evaluation and Sinks.

3.2 Core Components and Data Lifecycle

The system processes data through a strict, unidirectional lifecycle. The primary components include the **JSON Validator Filter**, the **Batch Pipe**, and the **Rule Evaluation Filter**.

Every measurement follows this sequential path:

1. **Ingestion (JSON Validator Filter):** Raw JSON packets are parsed and validated against a predefined schema. If a packet is malformed or violates the schema, it is immediately discarded.
2. **Aggregation (Batch Pipe):** Validated packets are grouped into a discrete *Batch*. This grouping enables efficient bulk processing rather than handling individual events.
3. **Evaluation (Rule Evaluation Filter):** The batch is processed against two logic types:
   - **Stateless Logic:** Applied to individual values within the current batch.
   - **Stateful Logic:** Compares current data at T_n with historical data from T_{n-1}.
4. **The Sink:** Based on the evaluation, data is sent to specific output streams: `valid_data.csv` for nominal states or `alarms.log` for anomalies.

3.3 Optimization Strategies

To improve the performance, the architecture implements parallelism at three distinct layers.

3.3.1 Data Partitioning (Cluster-Level Parallelism)

Since measurements are equally spaced and synchronized, data can be partitioned across the cluster. Using the **SLURM** scheduler, the system spins up multiple worker processes.

- **Strategy:** While *Worker A* evaluates a specific batch, *Worker B* simultaneously processes the subsequent batch.
- **State Management:** To maintain consistency for stateful rules, the state (values at T_{n-1}) is passed between workers during the hand-off.

3.3.2 Rule Concurrency (Task-Level Parallelism)

Evaluating a large volume of monitoring rules sequentially can create a computational bottleneck. Instead, these tasks are parallelized using multi-threading.

- **Independence:** Because $Rule_1$ (e.g., Temperature) does not depend on $Rule_2$ (e.g., Pressure), they are evaluated on separate threads concurrently.
- **Synchronization:** Logical Correlation Rules (e.g., $Rule_4$) act as synchronization points; they wait for the boolean results of their parent rules before finalizing their own output.

3.3.3 Vectorization (Hardware-Level Parallelism)

To enable hardware-level parallelism, incoming sensor data cannot be maintained in standard object structures (**Array of Structures**). Instead, specific data fields are extracted from the objects and explicitly reorganized into contiguous memory blocks, effectively transitioning the data layout into a **Structure of Arrays (SoA)**.

- **SIMD Utilization:** This precise memory alignment allows the cluster to utilize **Single Instruction, Multiple Data (SIMD)** instructions. Because values of the same type are now stored contiguously, the system can compare entire columns of sensor data against a threshold in a single clock cycle, significantly increasing throughput compared to iterative loops.

3.4 Sequence diagrams

The behaviour of the AstraLog-HPC system is illustrated through the following sequence diagrams, which detail the primary data pipeline and the internal mechanics of the concurrent rule evaluation.

Figure 4 illustrates the lifecycle of a measurement. Raw JSON packets transmitted by the sensors are first intercepted by the JSON Validator Filter. Validated data is subsequently aggregated into discrete batches by the Batch Pipe and dispatched to the Rule Evaluator Filter. Within this filter, both stateless and stateful logic are applied before routing the evaluated batch to the appropriate output stream.

Figure 5 shows the internal execution sequence of the Rule Evaluator Filter during concurrent processing. Following memory realignment into a Structure of Arrays (SoA) to enable SIMD utilization, independent rules are dispatched to separate threads based on their designated priority levels. To manage inter-rule dependencies, a Logical Correlator acts as a synchronization barrier, awaiting the boolean outputs of its parent rules. Once all required dependencies are resolved, the correlator finalizes the evaluation and sends the data to either the standard output CSV or the anomaly log.

3.5 Critical points and design decisions

3.5.1 Early data dropping

If a packet has malformed syntax or is missing mandatory fields, it is thrown away immediately.

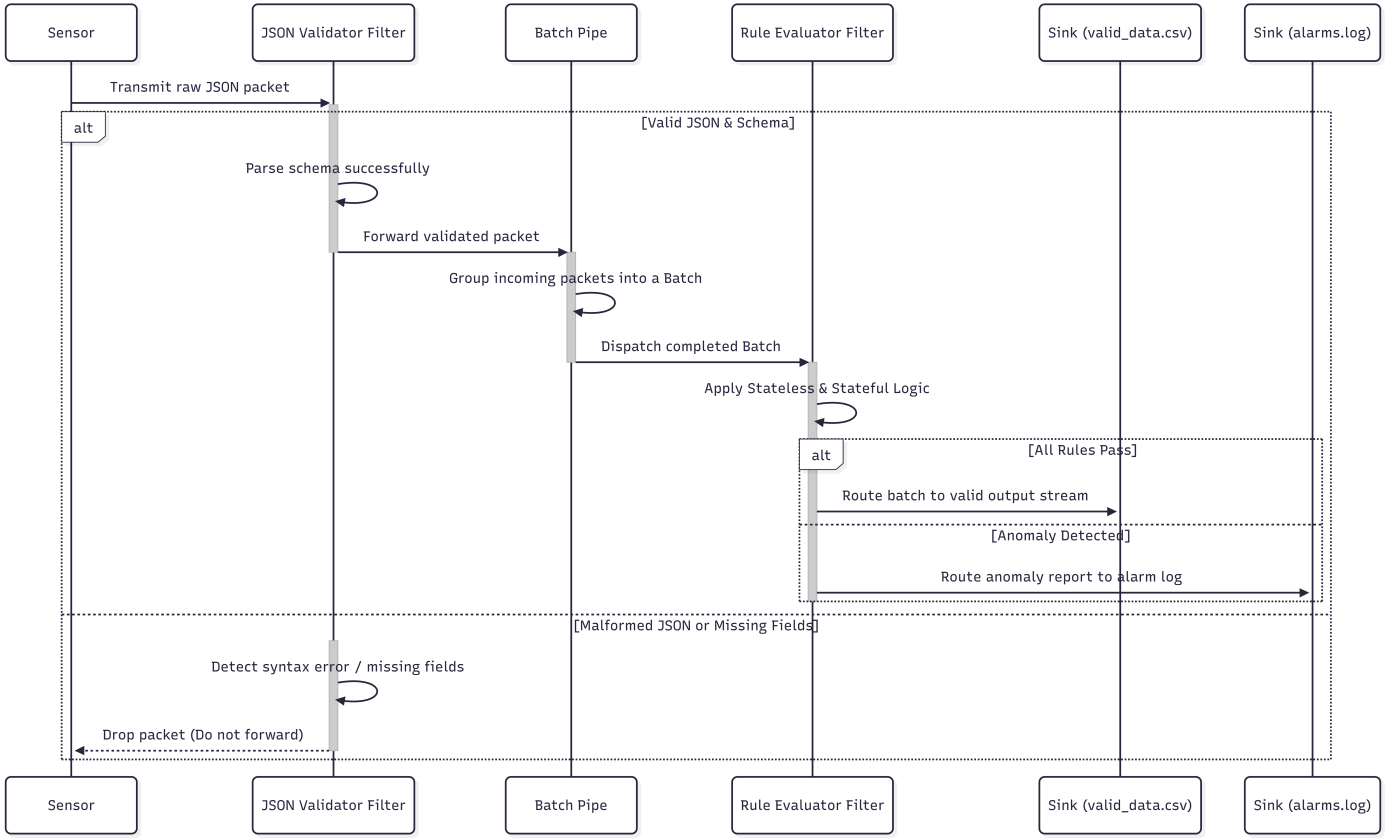

Figure 4: The Standard Data Pipeline: Validation, Batching, and Evaluation.

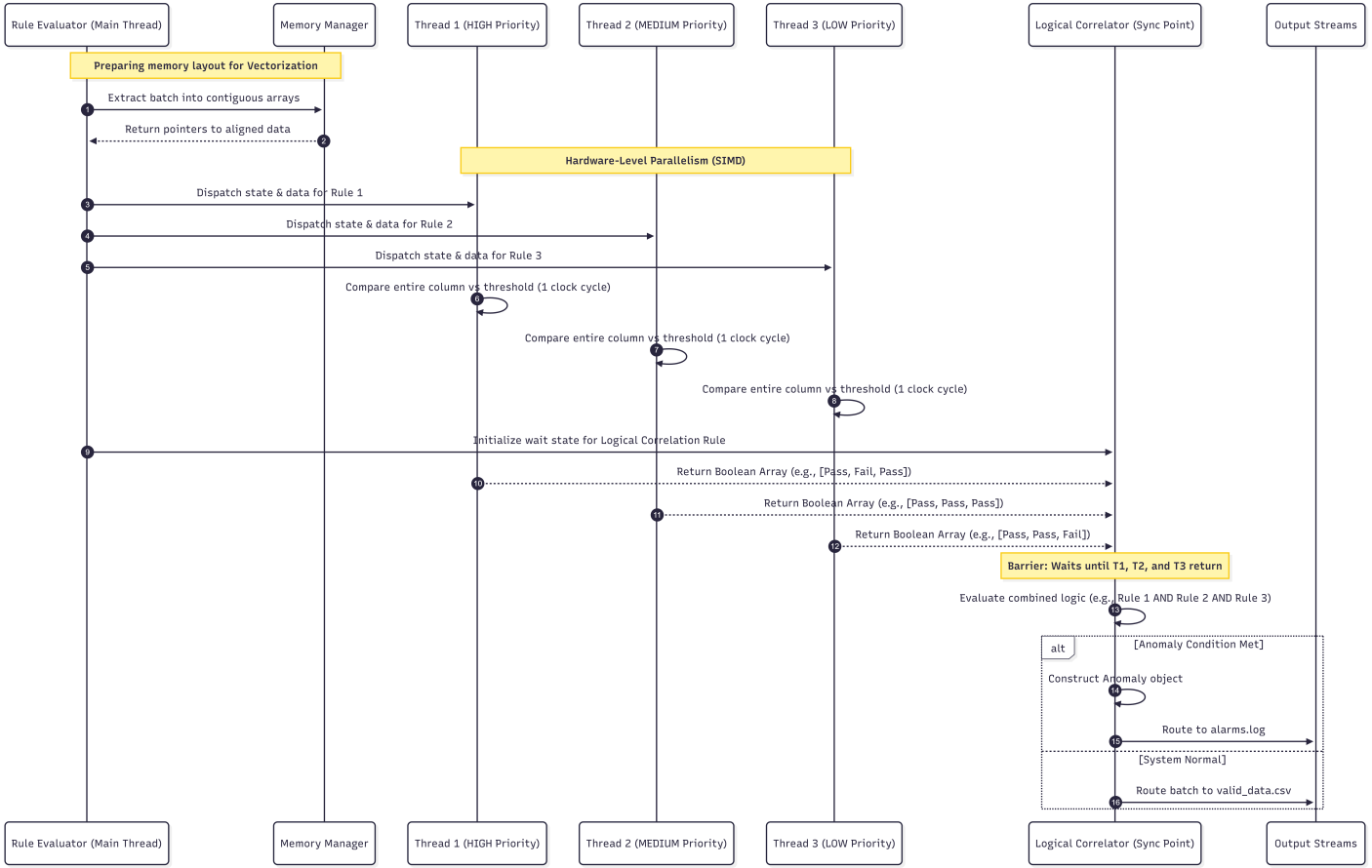

Figure 5: Parallel Rule Evaluation and Thread Synchronization.

3.5.2 State management across workers

One of the main challenges was dealing with stateful rules while processing data in parallel. Because different workers handle different batches, we have to pass the state, like the values at T_{n-1}, from one worker to the next. We decided only hand off the exact variables needed for the next batch instead of copying over large chunks of history.

3.5.3 Memory Layout for vectornization

To enable hardware-level parallelism, data cannot be maintained in standard object structures (**Array of Structures**). Instead, values must be extracted and stored in contiguous memory arrays. To achieve this, data fields are separated from individual objects and explicitly reorganized into a **Structure of Arrays**.

3.5.4 Rule synchronization

Because independent rules are evaluated simultaneously on separate threads, rule dependencies require explicit synchronization. Logical correlation rules are implemented as wait points that do not execute until their parent rules have returned a result.

4 Work distribution and effort

4.1 Work distribution

- **Problem Analysis**
  Daniela Hu managed the problem analysis, which includes understanding the requirements and constraints of the project, as well as identifying the key challenges and potential solutions.
  A first step is the summery of the project description, which is the first chapter of the document. Then the time scheduling and content planning, which means dividing the work into different parts and assigning them to different team members. Finally, the representation of the problem through the use of diagrams, such as the class diagram and the use case diagram, which are shown in the previous chapters.

- **Requirement analysis**
  Emiliano Liao managed the requirement analysis, which involves gathering and documenting the functional and non-functional requirements of the system.
  This includes identifying the relevant human and non-human actors, defining the use cases, making domain assumptions, and specifying the requirements in detail. The requirement analysis is crucial for ensuring that the design and implementation of the system meet the needs of the stakeholders and align with the project goals.

- **Design**
  Ji Junkai managed the design phase, which includes creating the architectural blueprint and detailed design specifications.
  This involves defining the overall architecture of the system, identifying the core components and their interactions, and designing the data flow and processing logic. The design phase also includes making critical decisions about optimization strategies and handling potential challenges in the implementation.

- **Work Review**
  Last but not least, we all contributed to the work review, which involves reviewing each other's work to ensure the quality and consistency of the project. This includes checking for errors, providing feedback, and making necessary revisions to improve the overall quality of the document and the project.
  Another important actor of this phase is AI, further details about its role will be explained in the next chapter.

Of course, the work distribution is not rigid and there is a lot of collaboration and communication among team members throughout the project. We all contribute to each phase of the project and support each other to ensure the success of the project.

Here below is a summary of the work distribution and the effort of each team member:

Team Member	**Part**	**Effort (hours)**
Daniela Hu	Project Planning & Diagrams	10
Ji Junkai	Design	15
Emiliano Liao	Requirements	12
All	Work Review	8

5 Usage of AI

As mentioned before, we have used AI for to finish our work. This choice is mainly orientated to the completeness of the project and not aimed to be substitution to human work, but as analysis of with deeper and wider considerations beyond our limitations.
The model used for our work are: Gemini 3.1 Pro, Chatgpt Pro and Copilot. Our goal is understanding our work limits, so using more models, we can compare the answer and distinguish the common points and the differences, which can be a good way to understand the problem more deeply.

5.1 Gemini 3.1 Pro

Work completeness and quality check
We have used Gemini 3.1 Pro to check the completeness and quality of our work, which is a crucial step in ensuring that our project meets the required standards and is free of errors.
In this case, Gemini 3.1 Pro find out some missing points in our work, such as the lack of component diagrams, absence of a detailed work distribution(effort table) and mistakes in our use cases diagram.
Error fixing
After identifying the missing points, we have used Gemini 3.1 Pro to fix the errors and improve our work.
For example, we have asked for the reasons of the mistakes in our use cases diagram and we have received a detailed explanation of the errors, which helped us to understand the problem and make the necessary corrections.
Furthermore, we have also used Gemini 3.1 Pro to improve our understading in the these kind of diagrams, to finally produce a correct and complete use cases diagram, which is shown in the previous chapter.

5.2 Chatgpt Pro

Work completeness and quality check
We have also used Chatgpt Pro to check the completeness and quality of our work, which confirms the findings of Gemini 3.1 Pro and provides additional insights.
This model displays a more detailed list of innacuracies in our work, such as word choices, grammar mistakes and some missing points in the requirement analysis.
Error fixing
After identifying the errors, we have used Chatgpt Pro to fix them and improve our work and, as with Gemini 3.1 Pro, we have also used it to improve our understanding of the use cases diagrams and the concepts involved, which has helped us to produce a better work.

5.3 Conclusions

Overall, the use of AI has been instrumental in enhancing the quality and completeness of our project.
By leveraging the capabilities of Gemini 3.1 Pro and Chatgpt Pro, we have been able to identify and rectify errors, improve our understanding of complex concepts, and ensure that our work meets the required standards.
And last but not least, the use of AI has also allowed us to save time and effort, facilitating the text writing process with the grammar check and latex language support.